

Aiuto v2 — 실행 일정

목적: V2 완성이 아니라 **면접에서 설명 가능한 핵심 줄기 완성**. 최종 목표는 취업이고 V2는 수단이다. 이 문서는 1페이지를 넘기지 않는다. 갱신: 매주 월요일 + 결정이 바뀔 때. 최종 갱신: 2026-08-25

현재 위치

STEP 5 인증  2026-07-29 · STEP 6 LLM 추상화  2026-08-15 (Gemini 단독)

STEP 7 (Celery + /ai/ingest)  진행 중 — 학습 단계

마감: 2026-09-06 (캡틴 지정) · 잔여 실가용 10일

STEP 7 목표 — 정면 목표 (2026-08-25 재정의)

자연어 입력 → 202 응답 → 워커가 받아 처리 → 레코드 1건 생성 → 폴링으로 확인

4단계(Celery 재시도 정책)는 선택 항목이다. 시간이 남으면 하고, 안 남으면 V3. 근거: 잔여 구현 시간이 4단계까지 감당하지 못하고, 재시도 설계 근거는 [STEP6_TROUBLESHOOTING.md](#)에 이미 문서로 남아 있어 코드 없이도 면접 방어가 된다.

단계 분할

단계	내용	게이트
1	도메인 모델 4종 + 워커 전용 동기 세션 + Alembic	동기 세션으로 ai_job INSERT/SELECT 성공
2	Celery 골격 + compose worker 서비스	더미 태스크 enqueue → 워커 수신·실행 로그 확인
3	분류 태스크 + ingest·폴링 엔드포인트	자연어 → 202 → 폴링 → done + 실제 레코드 1건
4	Celery 재시도 + 실패 경로 (선택)	Transient만 재시도, Permanent 즉시 실패

1·2단계를 반드시 분리한다. 동시에 하면 DB 문제인지 Celery 문제인지 분간이 안 된다.

일정 (8/25 ~ 9/6)

하루 Aiuto 배분: 오전 2.5h + 오후 1.5h (주간보드 기본 틀)

날짜	내용	산출
8/25 화	학습 B(아키텍처 4요소) + 학습 A(왜 Celery인가)	—
8/26 수	학습 C·D → 체크포인트1 학습요약 제출 + 1단계 설계 상담	설계 확정
8/27 목	1단계 구현 — 모델 4종 · 동기 세션 · Alembic	파일별 커밋

날짜	내용	산출
8/28 금	일정 소실	—
8/29 토	일정 소실	—
8/30 일	1단계 마무리 + 검증	게이트 ①
8/31 월	학습 F(워커 프로세스·세션·컨테이너) + 2단계 착수	—
9/1 화	2단계 마무리	게이트 ② · ★축소 판정일
9/2 수	학습 E(먹등성 부분만) + 3단계 착수 — worker/tasks.py	—
9/3 목	ai_job_service + ingest·폴링 엔드포인트	—
9/4 금	엔드투엔드 디버깅	—
9/5 토	게이트 ③ + 커밋 직전 전체 재실행	게이트 ③
9/6 일	STEP7_TROUBLESHOOTING.md + README 갱신 + 캡틴 보고	STEP 7 종료

★ 9/1 축소 판정: 게이트 ②가 9/1 안에 안 뚫리면 3단계에서 폴링 엔드포인트를 잘라내고, 워커가 레코드를 생성하는 것까지만 확인한다(조회는 DB 직접 확인으로 대체). Aiuto 총괄이 그날 집행한다.

학습 배치 원칙: A~D는 선행, E·F는 해당 구현 착수 직전에 붙인다. 워커가 눈앞에 돌기 전에 배우면 붙지 않는다.

진행 방식

학습(Gemini, 블록마다 새 창)

- [체크포인트1] 학습 요약을 총괄창에 제출 → 이해도 확인
- 본인이 직접 구현
- [체크포인트2] Claude Code 검증 — 코드 리뷰 · 에러 진단 · 테스트 실행
- 게이트 실행 확인
- 트러블슈팅 + 결과 요약을 총괄창에 제출 → 리뷰

- 코드는 본인이 직접 작성한다. 다음 STEP 코드를 미리 받지 않는다.
- 정답지는 막혔을 때만 연다. 막힘 규칙: 2일 → 총괄 보고 / 3일 → 정답지 개봉 허용
- Gemini의 확신에 찬 설명도 공식 문서·실행 결과로 교차 검증한다.
- 파일 단위로 학습 → 구현 → 실행검증 → 커밋. 전체를 만든 뒤 한 번에 검증하지 않는다.
- Git 브랜치·커밋 규칙은 각 단계 착수 시점에 총괄창이 그 단계 것만 지시한다.

절대규칙 — V2 전 구간

1. API = `asyncpg` 비동기 / Celery 워커 = `psycopg` 동기. `app/worker/` 아래에 `AsyncSession` · `AsyncOpenAI` · `async def` 전부 금지.
2. UTC · UUID PK · 모든 쿼리에 `user_id` 필터.
3. 모델 문자열 하드코딩 금지 — `config.py` + `.env`로만 주입. 현재 확정값 `gemini-3.5-flash-lite` / `thinking`은 문자열 `thinking_level="minimal"` (`gemini-2.5-flash`는 2026-10-16 종료 — 신규 고정 금

지)

4. 넓은 예외 캐치 금지. `except Exception` 대신 예상 가능한 타입만.
5. SQLAlchemy 2.0 스타일만. `Mapped / mapped_column / select()`.
6. LangChain / LangGraph 사용 금지 — Protocol이 추상화 계층이다.
7. 결정론적 연산은 LLM이 아니라 코드가 한다 (STEP 6 실측)
 - 날짜 앵커는 파이썬이 미리 계산해 프롬프트에 주입
 - 모델은 naive 로컬 시각만 출력, UTC 변환은 `ZoneInfo`로 코드가 처리
8. 재시도 계층을 섞지 않는다
 - 클라이언트 내부 재시도: 없음 / 폴백 체인: 1회 순회만
 - 태스크 재시도: Celery 담당, `autoretry_for`에 `TransientLLMError`만
9. 검증은 "성공 신호"가 아니라 "실제 값"을 본다 (STEP 5-6에서 5회 반복된 함정)
 - 예외가 안 뜨는 것 ≠ 정상 동작 · 필드가 존재하는 것 ≠ 값이 올바름
 - 같은 파일을 두 번째 편집할 때는 이전 수정이 남아 있는지 먼저 확인한다
 - 커밋 직전 전체 재실행
10. V3 이연 스코프 침범 금지 — refresh 토큰 · logout · 블랙리스트 · UserStats · `PATCH /users/me` · 수동 CRUD(POST/PATCH/DELETE) · 캘린더 · streak · `/stats` · Ollama · NVIDIA 폴백 · 상대시각 지원(dateparser)
11. STEP 완료 전 다른 프로젝트로 이탈 금지.

STEP 7 함정 — 미리 알고 갈 것

함정

- | | |
|---|---|
| 1 | 워커에 <code>AsyncSession</code> 이 딸려 들어옴 → 워커용 세션 팩토리는 이름부터 다르게 |
| 2 | Celery 태스크 인자에 ORM 객체를 넘기면 안 됨 → UUID 문자열만 넘기고 워커가 재조회 |
| 3 | 재시도가 곱해짐 (Celery × 폴백 체인) → <code>max_retries</code> 정할 때 총 호출 수 계산 |
| 4 | 워커가 환경변수를 못 읽음 → Compose worker 서비스에도 <code>env_file</code> 지정 |
| 5 | 컨테이너 타임존이 UTC → 시각 변환을 컨테이너 안에서 한 번 확인 |
| 6 | 태스크는 두 번 실행될 수 있다(at-least-once) → 레코드 중복 생성 방어 필요 |

남은 STEP

STEP	내용	상태
8	저장 + GET 조회	예정
9	최소 프론트	Swagger로 대체 — 구현하지 않음

STEP	내용	상태
10	EC2 배포	범위 밖 — 제안하지 않음

문서 지도

문서	위치	공개
SCHEDULE.md (이 문서)	메인 폴더	내부
CLAUDE.md	레포 루트	내부
STEP5_CHECKLIST.md · STEP6_TROUBLESHOOTING.md	메인 폴더	내부
STEP{n}_GEMINI_PROMPT.md (진행 중인 STEP만)	메인 폴더	내부
archive/	완료 STEP 자료	내부
README.md · docs/TROUBLESHOOTING.md	레포	공개

AIUTO_MASTER.md는 섹션 6·7·10이 구설계다. 참조하지 않는다.